

Machine Learning (Tiếng Việt)

Tom Mitchell — bản dịch chỉnh sửa

Ngày 3 tháng 7 năm 2026

Mục lục

1	Các training example của Bảng 3.2 được thể hiện trên Hình 3.1.	6
1.1	Giả thuyết tìm kiếm không gian trong Decision Tree	6
1.2	Học hỏi	6
1.3	Xu hướng quy nạp trong học tập Decision Tree	6
1.4	Các vấn đề trong việc học Decision Tree	7
1.5	Nhiệt độ: 40 48 60 72 80 90	9
1.6	PlayTennis: Không Không Có Có Có NO	9
1.7	Chi phí (A)	9
1.8	Tóm tắt và đọc thêm	9
2	Bài tập	10
2.1	Tài liệu tham khảo	10
2.2	C M 3 Decision Tree Học 79	10
2.3	Giới thiệu	11
2.4	Các vấn đề phù hợp để học Neural Network	12
3	1 1 30 Đầu ra	12
3.1	Perceptron	12
4	Khởi tạo mỗi Awi về 0.	14
4.1	Multi-layer network và thuật toán Backpropagation	14
4.2	Đầu giấu 4 ai sẽ trùn đầu	14
5	Xấu . Giấu	14
5.1	B~c~~~~o~~GATIO~(trainingaxamp~es, q, ni, , no,, , nhidden)	14
5.2	Nhận xét về thuật toán Backpropagation	15
5.3	Đầu vào Đầu ra Đầu vào	16
5.4	Ẩn giấu	16
5.5	Giá trị	16
5.6	Đầu ra	16
6	1000000 00 100000 000 10000 0000 1000 00000 100 000000 10 0000000	
1		16
7	5000 1o00 15000 20000	17

8	Lo00 2000 3000 4000 5000 6000	17
9	Machine Learning	18
9.1	Một ví dụ minh họa: Nhận dạng khuôn mặt	18
9.2	Weight mạng sau 100 lần lặp qua mỗi mẫu training	18
9.3	Các chủ đề nâng cao về mạng nơ ron nhân tạo	19
9.4	Tôi 120 Machine Learning	19
9.5	Tóm tắt và đọc thêm	20
10	Bài tập	20
10.1	Tài liệu tham khảo	21
10.2	Đánh giá giả thuyết	22
10.3	Động lực	22

Thuộc tính nào là classification tốt nhất?

S: [9+,5-I

Độ ẩm

Cao

[3+,4-I [6t,1-1

E S.985 E S.592

Tăng (S, Hurnidir)

S: [9+,5-I

E S.940 wx Mạnh mẽ

[6+,2-I [3+,3-I

Tăng (S, Gió)

0.0.1 Một ví dụ minh họa

Để minh họa hoạt động của ID3, hãy xem xét nhiệm vụ học tập được thể hiện bằng các ví dụ training trong Bảng 3.2. Ở đây, thuộc tính đích PlayTennis, có thể có các giá trị có hoặc không cho các buổi sáng Thứ Bảy khác nhau, sẽ được dự đoán dựa trên các thuộc tính khác của buổi sáng được đề cập. Hãy xem xét bước đầu tiên thông qua

Ngày dự báo Nhiệt độ Độ ẩm Gió PlayTennis

D1 Nắng Nóng Cao Yếu Không

D2 Nắng Nóng Cao Mạnh Không

D3 U ám Nóng Cao Yếu Có

D4 Mưa Nhẹ Cao Yếu Có

D5 Mưa Mát Bình thường Yếu Có

D6 Mưa Mát Bình thường Mạnh Không

D7 U ám Mát Bình thường Mạnh Có

D8 Nắng Nhẹ Cao Yếu Không

D9 Nắng Mát Bình thường Yếu Có

D10 Mưa Nhẹ Bình thường Yếu Có

D11 Nắng Nhẹ Bình thường Mạnh Có

D12 U ám Nhẹ Cao Mạnh Có

D13 U ám Nóng Bình thường Yếu Có

D14 Mưa Nhẹ Cao Mạnh Không

TABLE 3.2 Ví dụ training cho khái niệm mục tiêu PlayTennis.

Thuật toán trong đó nút trên cùng của decision tree được tạo. Thuộc tính nào cần được kiểm tra đầu tiên trong cây? ID3 xác định mức tăng thông tin cho từng thuộc tính ứng cử viên (ví dụ: Outlook, Nhiệt độ, Độ ẩm và Gió), sau đó chọn thuộc tính có mức tăng thông tin cao nhất. Việc tính toán độ lợi thông tin cho hai thuộc tính này được thể hiện trong Hình 3.3. Các giá trị thu được thông tin cho cả bốn thuộc tính là

1 Các training example của Bảng 3.2 được thể hiện trên Hình 3.1.

1.1 Giả thuyết tìm kiếm không gian trong Decision Tree

1.2 Học hỏi

Giống như các phương pháp inductive learning khác, ID3 có thể được mô tả là tìm kiếm một không gian giả thuyết phù hợp với các ví dụ training. Hypothesis space được ID3 tìm kiếm là tập hợp decision trees có thể có. ID3 thực hiện tìm kiếm leo đồi từ đơn giản đến phức tạp thông qua hypothesis space này, bắt đầu từ cây trống, sau đó xem xét các giả thuyết phức tạp hơn dần dần để tìm kiếm decision tree classification chính xác dữ liệu training. Chức năng đánh giá

$\{D1, D2, \dots, D141$

$P+S-I$

Thuộc tính nào cần được kiểm tra ở đây?

FIGURE 3.4 decision tree đã học được một phần từ bước đầu tiên của ID3. Các ví dụ training được sắp xếp theo các nút con tương ứng. Hậu duệ Overcast chỉ có các ví dụ tích cực và do đó trở thành nút lá với classification Có. Hai nút còn lại sẽ được mở rộng hơn nữa, bằng cách chọn thuộc tính có mức tăng thông tin cao nhất so với các tập hợp con mẫu mới.

Hướng dẫn tìm kiếm leo đồi này là thước đo thu được thông tin. Việc tìm kiếm này được mô tả trong Hình 3.5.

1.3 Xu hướng quy nạp trong học tập Decision Tree

Chính sách mà ID3 khái quát hóa từ các ví dụ training được quan sát để classification các trường hợp không nhìn thấy là gì? Nói cách khác, inductive bias của nó là gì? Nhớ lại Chương 2 rằng inductive bias là tập hợp các giả định, cùng với dữ liệu training, chứng minh một cách suy diễn các classification mà người học chỉ định cho các trường hợp trong tương lai.

Inductive bias gần đúng của ID3: Cây ngắn hơn được ưa thích hơn cây lớn hơn.

Gần đúng hơn với inductive bias của ID3: Cây ngắn hơn được ưa thích hơn cây dài hơn. Những cây có thuộc tính thu được thông tin cao gần gốc sẽ được ưu tiên hơn những cây không có.

1.3.1 Những thành kiến hạn chế và những thành kiến ưu tiên

Có một sự khác biệt thú vị giữa các loại inductive bias được thể hiện bởi ID3 và bởi thuật toán CANDIDATE-ELIMINATION được thảo luận trong Chương 2.

1.3.2 Tại sao lại thích những giả thuyết ngắn?

Inductive bias của ID3 ưu tiên decision trees ngắn hơn có phải là cơ sở hợp lý để khái quát hóa ngoài dữ liệu training không? Các triết gia và những người khác đã tranh luận về câu hỏi này trong nhiều thế kỷ và cuộc tranh luận này vẫn chưa được giải quyết cho đến ngày nay. William xứ Occam là một trong những người đầu tiên thảo luận về vấn đề này vào khoảng năm 1320, vì vậy bias này thường được gọi là dao cạo của Occam.

Dao cạo của Occam: Ưu tiên giả thuyết đơn giản nhất phù hợp với dữ liệu.

Tại sao chúng ta nên tin rằng tập hợp nhỏ các giả thuyết bao gồm decision trees với các mô tả ngắn gọn sẽ phù hợp hơn vô số các tập hợp giả thuyết nhỏ khác mà chúng ta có thể định nghĩa?

~ ~ p r e n t l ~ trong khi cạo râu.

Đúng đối với các trường hợp được decision tree classification dương tính trong Hình 3.1 và sai trong các trường hợp khác. Do đó, hai người học, đều áp dụng dao cạo của Occam, sẽ khái quát hóa theo những cách khác nhau nếu một người sử dụng thuộc tính XYZ để mô tả các ví dụ của nó và người kia chỉ sử dụng các thuộc tính Outlook, Nhiệt độ, Độ ẩm và Gió.

Hiện tại, chúng ta tạm dừng cuộc tranh luận về dao cạo của Occam. Chúng ta sẽ xem lại nó trong

Chương 6, nơi chúng ta thảo luận về nguyên tắc Độ dài mô tả tối thiểu, một phiên bản của dao cạo Occam có thể được diễn giải trong khuôn khổ Bayesian.

1.4 Các vấn đề trong việc học Decision Tree

Các vấn đề thực tế khi học decision trees bao gồm xác định mức độ phát triển decision tree, xử lý các thuộc tính liên tục, chọn thước đo lựa chọn thuộc tính thích hợp, xử lý dữ liệu training với các giá trị thuộc tính bị thiếu, xử lý các thuộc tính với chi phí khác nhau và cải thiện hiệu quả tính toán. Dưới đây chúng ta thảo luận về từng vấn đề và phần mở rộng của thuật toán ID3 cơ bản để giải quyết chúng. Bản thân ID3 đã được mở rộng để giải quyết hầu hết các vấn đề này và kết quả là hệ thống được đổi tên thành C4.5 (Quinlan 1993).

1.4.1 Tránh trang bị quá nhiều dữ liệu

Thuật toán được mô tả trong Bảng 3.1 phát triển từng nhánh của cây vừa đủ sâu để classification hoàn hảo các ví dụ training. Mặc dù điều này đôi khi là một

Chiến lược hợp lý, trên thực tế nó có thể dẫn đến khó khăn khi có nhiều trong dữ liệu hoặc khi số lượng mẫu training quá nhỏ để tạo ra một mẫu đại diện cho hàm mục tiêu thực sự. Trong cả hai trường hợp này, thuật toán đơn giản này có thể tạo ra các cây vượt trội hơn các mẫu training.

Kích thước của cây (số nút)

FIGURE 3.6 Overfitting trong decision tree learning. Khi ID3 thêm các nút mới để phát triển decision tree, độ chính xác của cây được đo qua các ví dụ training sẽ tăng lên một cách đơn điệu. Tuy nhiên, khi đo trên một tập hợp các mẫu thử nghiệm độc lập với các mẫu training, độ chính xác ban đầu tăng lên, sau đó giảm xuống. Phần mềm và dữ liệu để thử nghiệm các biến thể của cốt truyện này có sẵn trên World Wide Web tại <http://www.cs.cmu.edu/~torn/mlbook.html>.

Việc xây dựng thêm cây sẽ làm giảm độ chính xác của nó đối với các mẫu thử nghiệm mặc dù độ chính xác của nó trên các mẫu training đã tăng lên.

Với dữ liệu ban đầu không có lỗi, ID3 tạo ra decision tree như trong Hình 3.1. Tuy nhiên, việc bổ sung ví dụ không chính xác này bây giờ sẽ khiến ID3 xây dựng một cây phức tạp hơn. Cụ thể, ví dụ mới sẽ được sắp xếp vào nút lá thứ hai từ bên trái trong cây đã học của Hình 3.1, cùng với các ví dụ tích cực trước đó là D9 và D1 1. Vì ví dụ mới được gán nhãn là ví dụ tiêu cực, ID3 sẽ tìm kiếm các sàng lọc tiếp theo cho cây bên dưới nút này. Tất nhiên, miễn là ví dụ sai sót mới khác với các ví dụ khác được liên kết với nút này theo một cách tùy ý nào đó, ID3 sẽ thành công trong việc tìm ra thuộc tính quyết định mới để tách ví dụ mới này khỏi hai ví dụ tích cực trước đó tại nút cây này.

thực hiện bằng cách xác định động các thuộc tính có giá trị rời rạc mới để phân chia giá trị thuộc tính liên tục thành một tập hợp các khoảng rời rạc. Cụ thể, đối với thuộc tính A có giá trị liên tục, thuật toán có thể tự động tạo thuộc tính boolean mới A, điều đó đúng nếu $A < c$ và sai nếu ngược lại. Câu hỏi duy nhất là làm thế nào để chọn giá trị tốt nhất cho ngưỡng c.

Ví dụ: giả sử chúng ta muốn bao gồm thuộc tính có giá trị liên tục

1.5 Nhiệt độ: 40 48 60 72 80 90

1.6 PlayTennis: Không Không Có Có Có NO

1.6.1 Các biện pháp thay thế để lựa chọn thuộc tính

Có một bias tự nhiên trong thước đo mức tăng thông tin ưu tiên các thuộc tính có nhiều giá trị hơn những thuộc tính có ít giá trị. Một ví dụ điển hình, hãy xem xét thuộc tính Ngày, có số lượng giá trị có thể rất lớn (ví dụ: ngày 4 tháng 3,

Tăng (S, A) GainRatio(S, A) r Phân tách thông tin(S, A)

Lưu ý rằng thuật ngữ SplitInfomzation không khuyến khích việc lựa chọn các thuộc tính có nhiều giá trị được phân bố đồng đều. Ví dụ: hãy xem xét một tập hợp n ví dụ được phân tách hoàn toàn bởi thuộc tính A (ví dụ: Ngày). Trong trường hợp này, giá trị SplitInfomzation sẽ là log, n. Ngược lại, thuộc tính boolean B chia đôi chính xác n ví dụ giống nhau sẽ có Hệ số phân tách là 1. Nếu thuộc tính A và B tạo ra mức tăng thông tin giống nhau thì rõ ràng B sẽ đạt điểm cao hơn theo thước đo Tỷ lệ khuếch đại.

3.7.4 Xử lý các ví dụ training bị thiếu giá trị thuộc tính

Trong một số trường hợp nhất định, dữ liệu có sẵn có thể thiếu giá trị cho một số thuộc tính. Ví dụ: trong lĩnh vực y tế mà chúng ta muốn dự đoán bệnh nhân outcome dựa trên các xét nghiệm khác nhau trong lab, có thể kết quả xét nghiệm máu lab chỉ có sẵn cho một nhóm nhỏ bệnh nhân. Trong những trường hợp như vậy, estimate thường có giá trị thuộc tính bị thiếu dựa trên các ví dụ khác mà thuộc tính này có giá trị đã biết.

1.6.2 Xử lý các thuộc tính với chi phí khác nhau

Trong một số nhiệm vụ học tập, các thuộc tính thể hiện có thể có chi phí liên quan. Ví dụ: khi học cách classification các bệnh nội khoa, chúng ta có thể mô tả bệnh nhân theo các thuộc tính như Nhiệt độ, BiopsyResult, Mạch, BloodTestResults, v.v. Các thuộc tính này khác nhau đáng kể về chi phí, cả về chi phí tiền tệ và chi phí mang lại sự thoải mái cho bệnh nhân. Trong các nhiệm vụ như vậy, chúng ta ưu tiên decision trees sử dụng các thuộc tính chi phí thấp nếu có thể, chỉ dựa vào các thuộc tính chi phí cao khi cần để tạo ra các classification đáng tin cậy.

1.7 Chi phí (A)

2GaWS.A) - 1

1.8 Tóm tắt và đọc thêm

Các điểm chính của chương này bao gồm:

Decision tree learning cung cấp một phương pháp thực tế cho concept learning và để học các hàm có giá trị rời rạc khác. Nhóm thuật toán ID3 suy ra decision trees bằng

cách phát triển chúng từ gốc trở xuống, tham lam chọn thuộc tính tốt nhất tiếp theo cho mỗi nhánh quyết định mới được thêm vào cây. ID3 searches a complete hypothesis space (i.e., the space of decision trees can represent any discrete-valued function defined over discrete-valued instances). Do đó, nó tránh được khó khăn lớn liên quan đến các phương pháp chỉ xem xét các tập hợp giả thuyết hạn chế: hàm mục tiêu có thể không có trong hypothesis space. Inductive bias tiềm ẩn trong ID3 bao gồm ưu tiên cho các cây nhỏ hơn; nghĩa là, việc tìm kiếm của nó thông qua hypothesis space chỉ phát triển cây ở mức cần thiết để classification các mẫu training có sẵn. Overfitting dữ liệu training là một vấn đề quan trọng trong decision tree learning. Vì các ví dụ training chỉ là mẫu của tất cả các trường hợp có thể,

Để biết thêm chi tiết về cảm ứng decision tree, một cuốn sách xuất sắc của Quinlan (1993) thảo luận nhiều vấn đề thực tế và cung cấp mã thực thi cho C4.5. Mingers (1989a) và Buntine và Niblett (1992) cung cấp hai nghiên cứu thực nghiệm so sánh các biện pháp lựa chọn thuộc tính khác nhau. Mingers (1989b) và Malerba et al. (1995) cung cấp các nghiên cứu về các chiến lược cắt tỉa khác nhau. Các thí nghiệm so sánh decision tree learning và các phương pháp học tập khác có thể được tìm thấy trong nhiều bài báo, bao gồm (Dietterich và cộng sự 1995; Fisher và McKusick 1989; Quinlan 1988a; Shavlik và cộng sự. 1991; Thrun và cộng sự. 1991; Weiss và Kapouleas 1989).

2 Bài tập

Cho decision trees biểu diễn các hàm boolean sau: (a) $A \wedge \neg B$ (b) $A \vee [B \wedge A \wedge C]$ (c) $A \oplus B$ (d) $[A \wedge A] \vee [C \wedge A \wedge D]$ Hãy xem xét tập hợp các ví dụ training sau:

Ví dụ Classification a1 a2

(b) Mức thu được thông tin của a2 so với các ví dụ training này là bao nhiêu?

(b) Mỗi quan hệ giữa decision tree đã học và version space đã học là gì

(thể hiện trong Hình 2.3 của Chương 2) được học từ những ví dụ tương tự này? Cây đã học có tương đương với một trong các thành viên của version space không?

Dự báo nhiệt độ không khí trên bầu trời Độ ẩm gió nước Thường thức thể thao?

Nắng Ấm Bình thường Yếu Ấm Tương tự Không

2.1 Tài liệu tham khảo

Brodley, C. E., & Utgoff, P. E. (1995). Decision trees đa biến. *Machine Learning*, 19, 45-77. Buntine, W., & Niblett, T. (1992). Một so sánh sâu hơn về các quy tắc phân chia cho việc quy nạp cây quyết định.

Machine Learning, 8, 75-86.

Dietterich, T. G., Hild, H., & Bakiri, G. (1995). So sánh ID3 và BACKPROPAGATION cho

Ảnh xạ văn bản thành giọng nói tiếng Anh. *Machine Learning*, 18(1), 51-80.

2.2 C M 3 Decision Tree Học 79

Fisher, D. H. Và McKusick, K. B. (1989). So sánh thực nghiệm giữa ID3 và backpropagation.

Kỷ yếu Hội nghị chung quốc tế lần thứ 11 về AI (trang 788-793). Morgan Kaufmann.

Friedman, J. H. (1977). Quy tắc quyết định phân vùng đệ quy cho classification không tham số. *IEEE*

Giao dịch trên máy tính @p. 404408).

- Mingers, J. (1989a). Một so sánh thực nghiệm về các biện pháp lựa chọn cho việc tạo ra cây quyết định.
Machine Learning, 3(4), 319-342.
- Mingers, J. (1989b). Một so sánh thực nghiệm về các phương pháp cắt tỉa để tạo ra cây quyết định.
Machine Learning, 4(2), 227-243.
- Murthy, S. K., Kasif, S., & Salzberg, S. (1994). Một hệ thống cảm ứng decision trees xiên.
Tập chí Nghiên cứu Trí tuệ Artificial, 2, 1-33.
- Nunez, M. (1991). Việc sử dụng kiến thức nền tảng trong quy nạp decision tree.
Machine Learning,
- Quinlan, J. R. (1979). Khám phá các quy tắc bằng quy nạp từ tập hợp lớn các ví dụ. **Ổ D**
Michie (Ed.), Hệ thống chuyên gia trong thời đại vi điện tử. Đại học Edinburgh Nhấn.
- Quinlan, J. R. (1986). Cảm ứng decision trees. Machine Learning, 1(1), 81-106.
- Quinlan, J. R. (1987). Quy tắc quy nạp với dữ liệu thống kê - so sánh với nhiều regression.
Tập chí của Hiệp hội Nghiên cứu Hoạt động, 38, 347-352.
- Quinlan, JR (1988b). Decision trees và các thuộc tính đa giá trị. Trong Hayes, Michie và Richards
(Eds.), Máy thông minh 11, (trang 305-318). Oxford, Anh: Nhà xuất bản Đại học Oxford.
- Quinlan, JR, & Rivest, R. (1989). Thông tin và Tính toán, (go), 227-248.
- Quinlan, JR (1993). C4.5: Các chương trình dành cho Machine Learning. San Mateo, CA: Morgan Kaufmann.
- Rissanen, J. (1983). Một ưu tiên phổ quát cho số nguyên và ước tính theo độ dài mô tả tối thiểu.
Biên niên sử thống kê 11 (2), 416-431.
- Tân, M. (1993). Học kiến thức về classification với chi phí hợp lý và các ứng dụng của nó trong chế tạo robot.
Machine Learning, 13(1), 1-33.
- Utgoff, P. E. (1989). Cảm ứng tăng dần của decision trees. Machine Learning, 4(2), 161-186.
- Utgoff, P. E., & Brodley, C. E. (1991). Máy tuyến tính decision trees, (Báo cáo kỹ thuật COINS 91-10). Đại học Massachusetts, Amherst, MA.

2.3 Giới thiệu

2.3.1 Động lực sinh học

4.2 NEURAL NETWORK REPRESENTATIONS Một ví dụ điển hình về việc học ANN được cung cấp bởi hệ thống ALVINN của Pomerleau (1993), hệ thống này sử dụng ANN đã học để điều khiển phương tiện tự hành lái

Ở tốc độ bình thường trên đường cao tốc công cộng. Đầu vào của neural network là lưới cường độ pixel 30 x 32 thu được từ một camera hướng về phía trước gắn trên xe. Đầu ra của mạng là hướng mà xe được điều khiển. ANN được training để bắt chước các lệnh lái được quan sát của con người điều khiển phương tiện trong khoảng 5 phút. ALVINN đã sử dụng các mạng đã học của mình để lái xe thành công ở tốc độ lên tới 70 dặm một giờ và trong khoảng cách 90 dặm trên đường cao tốc công cộng (lái xe ở làn bên trái của đường cao tốc công cộng bị chia cắt, có sự hiện diện của các phương tiện khác).

2.4 Các vấn đề phù hợp để học Neural Network

Việc học ANN rất phù hợp với các vấn đề trong đó dữ liệu training tương ứng với dữ liệu cảm biến phức tạp, ồn ào, chẳng hạn như đầu vào từ máy ảnh và micro.

E2'

Thăng

Phía trước

3 1 1 30 Đầu ra

Võng mạc đầu vào cảm biến 30x32

1

~t cũng có thể áp dụng cho các bài toán thường sử dụng nhiều biểu diễn ký hiệu hơn, chẳng hạn như các nhiệm vụ decision tree learning đã thảo luận trong Chương 3. Trong những trường hợp này, ANN và decision tree learning thường tạo ra kết quả có độ chính xác tương đương. Xem Shavlik và cộng sự. (1991) và Weiss và Kapouleas (1989) để so sánh thử nghiệm việc học decision tree và ANN. Thuật toán BACKPROPAGATION là kỹ thuật học ANN được sử dụng phổ biến nhất. Nó phù hợp cho các vấn đề có các đặc điểm sau:

I Khả năng con người hiểu được chức năng mục tiêu đã học là không quan trọng. Các weight mà neural networks học được thường khó giải thích đối với con người. Neural networks đã học được khó truyền đạt tới con người hơn các quy tắc đã học được.

3.1 Perceptron

Một loại hệ thống ANN dựa trên một đơn vị gọi là perceptron, được minh họa trong

Học một perceptron bao gồm việc chọn các giá trị cho các weight $w_0, \dots, w_n$.

Do đó, không gian H của các giả thuyết ứng cử viên được xem xét trong quá trình học perceptron là tập hợp tất cả các vector weight có giá trị thực có thể có.

3.1.1 Sức mạnh đại diện của Perceptron

Chúng ta có thể xem perceptron như biểu diễn bề mặt quyết định siêu phẳng trong không gian n chiều của các thể hiện (tức là các điểm). Perceptron xuất ra giá trị 1 cho các trường hợp nằm ở một phía của siêu phẳng và xuất ra -1 cho các trường hợp nằm ở phía bên kia, như minh họa trong Hình 4.3. Phương trình cho siêu phẳng quyết định này là $w_0 + w_1x_1 + \dots + w_nx_n = 0$. Tất nhiên, một số tập hợp ví dụ dương và âm không thể được phân tách bằng bất kỳ siêu phẳng nào. Những tập hợp có thể tách rời được gọi là tập hợp mẫu có thể phân tách tuyến tính.

FIGURE 4.3 Một tri giác.

Perceptron có thể biểu diễn tất cả các hàm boolean nguyên thủy AND, OR,

NAND ($\neg$ AND) và NOR ($\neg$ OR). Tuy nhiên, thật không may, một số hàm boolean không thể được biểu diễn bằng một perceptron đơn lẻ, chẳng hạn như hàm XOR có giá trị là 1 khi và chỉ khi $x_1 \neq x_2$. Lưu ý tập hợp các ví dụ training không thể phân tách tuyến tính được hiển thị trong Hình 4.3(b) tương ứng với hàm XOR này.

FIGURE 4.3 Bề mặt quyết định được biểu thị bằng perceptron hai đầu vào. (a) Một tập hợp các ví dụ training và bề mặt quyết định của một perceptron classification chứng minh một

cách chính xác. (b) Một tập hợp các ví dụ training không thể phân tách tuyến tính (nghĩa là không thể classification chính xác theo bất kỳ đường thẳng nào). x_1 và x_2 là đầu vào Perceptron. Các ví dụ tích cực được biểu thị bằng "+", tiêu cực bằng "-".

Đầu vào được cung cấp cho nhiều đơn vị và đầu ra của các đơn vị này sau đó được đưa vào giai đoạn thứ hai, giai đoạn cuối cùng. Một cách là biểu diễn hàm boolean ở dạng phân biệt chuẩn (tức là, dưới dạng phân tách (OR) của một tập hợp các liên từ (ANDs) của các đầu vào và các phủ định của chúng). Lưu ý rằng đầu vào của một perceptron AND có thể bị phủ định đơn giản bằng cách thay đổi dấu của weight đầu vào tương ứng.

3.1.2 Quy tắc training Perceptron

Mặc dù chúng ta quan tâm đến mạng học tập của nhiều đơn vị được kết nối với nhau, nhưng chúng ta hãy bắt đầu bằng cách tìm hiểu cách học các weight cho một perceptron đơn lẻ. Ở đây, vấn đề học chính xác là xác định một vector weight làm cho perceptron tạo ra đầu ra f chính xác cho mỗi training examples đã cho.

Ví dụ này Lưu ý quy tắc training sẽ tăng w , trong trường hợp này, vì $(t - o)$,

3.1.3 Gradient Descent và Quy tắc Delta

Mặc dù quy tắc perceptron tìm thấy một vector weight thành công khi các mẫu training có thể phân tách tuyến tính, nhưng nó có thể không hội tụ nếu các mẫu không phân tách tuyến tính. Quy tắc training thứ hai, được gọi là quy tắc delta, được thiết kế để khắc phục khó khăn này. Nếu các mẫu training không thể phân tách tuyến tính thì quy tắc delta sẽ hội tụ về một xấp xỉ phù hợp nhất với khái niệm đích.

Do đó, một đơn vị tuyến tính tương ứng với giai đoạn đầu tiên của một perceptron, không có ngưỡng.

Chúng ta giả sử những điều này là cố định trong quá trình training, vì vậy chúng ta không bận tâm viết E như một hàm rõ ràng của những điều này. Chương 6 cung cấp cơ sở biện minh Bayes cho việc chọn định nghĩa cụ thể này của E . Cụ thể, ở đó chúng ta chỉ ra rằng trong những điều kiện nhất định, giả thuyết cực tiểu hóa E cũng là giả thuyết có khả năng xảy ra nhất trong H dựa trên dữ liệu training.

4.4.3.1 VISUALIZING THE HYPOTHESIS SPACE

4.4.3.2 DERIVATION OF THE GRADIENT DESCENT RULE

Làm thế nào chúng ta có thể tính toán hướng đi xuống dốc nhất dọc theo bề mặt sai số?

Hướng này có thể được tìm bằng cách tính đạo hàm của E đối với từng thành phần của vector $\mathbf{2}$. Đạo hàm vector này được gọi là gradient của E đối với $\mathbf{2}$, được viết là $\sim (i, r)$.

E . Do đó, giá trị âm của vector này cho biết hướng giảm mạnh nhất.

Ví dụ, mũi tên trong Hình 4.4 hiển thị gradient phủ định $-VE(G)$ cho một điểm cụ thể trong mặt phẳng w_0, w_1 .

4.4.3.3 STOCHASTIC APPROXIMATION TO GRADIENT DESCENT

Giảm dần độ dốc là một model chung quan trọng cho việc học. Đó là một chiến lược tìm kiếm thông qua hypothesis space lớn hoặc vô hạn có thể được áp dụng bất cứ khi nào (1) hypothesis space chứa các giả thuyết được tham số hóa liên tục (ví dụ: weight trong một đơn vị tuyến tính) và (2) lỗi có thể được phân biệt theo giả thuyết parameters này. Những khó khăn chính trong thực tế khi áp dụng giảm độ dốc là (1) việc hội tụ về mức cực tiểu cục bộ đôi khi có thể khá chậm (nghĩa là có thể yêu cầu hàng nghìn bước giảm

độ dốc) và (2) nếu có nhiều cực tiểu cục bộ trên bề mặt lỗi thì không có gì đảm bảo rằng quy trình sẽ tìm được mức tối thiểu toàn cục.

Mỗi ví dụ training là một cặp có dạng (x, t) , trong đó x là vector của các giá trị đầu vào và t là giá trị đầu ra mục tiêu. Q là tốc độ học tập (ví dụ: 0,05). . Khởi tạo mỗi w thành một giá trị ngẫu nhiên nhỏ nào đó. Cho đến khi điều kiện kết thúc được đáp ứng, Do

4 Khởi tạo mỗi Awi về 0.

Với mỗi đơn vị trọng lượng tuyến tính w , Do

Với mỗi đơn vị trọng lượng tuyến tính w_i , Do

Trong đó t và o_d là giá trị đích và giá trị đầu ra đơn vị cho ví dụ training d . Stochastic gradient descent lặp lại trên các ví dụ training d trong D , tại mỗi lần lặp sẽ thay đổi các weight theo độ dốc đối với

Cả hai phương pháp giảm độ dốc ngẫu nhiên và tiêu chuẩn đều được sử dụng phổ biến trong thực tế.

4.0.1 Bình luận

Chúng ta đã xem xét hai thuật toán tương tự để học lặp lại các weight perceptron. Sự khác biệt chính giữa các thuật toán này là perceptron training

4.1 Multi-layer network và thuật toán Backpropagation

Chúng ta sẽ sử dụng loại đơn vị nào làm cơ sở để xây dựng mạng nhiều lớp? Lúc đầu, chúng ta có thể muốn chọn các đơn vị tuyến tính được thảo luận ở phần trước.

4.2 Đầu giấu 4 ai sẽ trùn đầu

5 Xấu . Giấu

FIGURE 4.6 Đơn vị ngưỡng sigmoid.

5.0.1 Thuật toán BACKPROPAGATION

Tôi nhắm mục tiêu và các giá trị đầu ra được liên kết với đơn vị đầu ra thứ k và ví dụ training d .

5.1 B~c~~~o~~GATIO~(trainingaxamp~es, q, ni, , no,, , nhidden)

Mỗi ví dụ training là một cặp có dạng (x, i) , trong đó x là vector của các giá trị đầu vào mạng và là vector của các giá trị đầu ra của mạng đích.

Q là tốc độ học tập (ví dụ: .05). N_i là số lượng đầu vào mạng, số lượng đơn vị trong lớp ẩn và không, số lượng đơn vị đầu ra. Đơn vị i đầu vào vào đơn vị j được ký hiệu là x_{ji} và weight từ đơn vị i đến đơn vị j được ký hiệu là w_{ji} .

A Tạo mạng chuyển tiếp nguồn cấp dữ liệu với n_i đầu vào, đơn vị ẩn trung gian và đơn vị đầu ra n_o . A Khởi tạo tất cả các weight mạng thành các số ngẫu nhiên nhỏ (ví dụ: trong khoảng -0,05 đến 0,05).

Truyền bá đầu vào chuyển tiếp qua mạng:

Truyền ngược các lỗi qua mạng:

2. Với mỗi đơn vị đầu ra mạng k , tính toán sai số S_k của nó

$6k - \text{được}(1 - o_k)(t_k - o_k)$

3. Với mỗi đơn vị ẩn h , tính sai số $6h$ của nó

Ồi.. $Jl - I$

Một chỉ mục (ví dụ: một số nguyên) được gán cho mỗi nút trong mạng, trong đó "nút" là đầu vào của mạng hoặc đầu ra của một số đơn vị trong mạng.

0 xji biểu thị đầu vào từ nút i đến đơn vị j và wji biểu thị tương ứng

I Thuật toán trong Bảng 4.2 cập nhật các weight tăng dần, theo

I Trình bày từng ví dụ training. Điều này tương ứng với một xấp xỉ ngẫu nhiên đối với việc giảm độ dốc. Để có được gradient thực sự của E , người ta sẽ tính tổng các giá trị 6 , x , trên tất cả các mẫu training trước khi thay đổi giá trị weight.

4.5.2.1 ADDING MOMENTUM

4.5.2.2 LEARNING IN ARBITRARY ACYCLIC NETWORKS

5.1.1 Đạo hàm của Quy tắc BACKPROPAGATION

Phần này trình bày nguồn gốc của quy tắc điều chỉnh weight BACKPROPAGATION.

Nó có thể được bỏ qua trong lần đọc đầu tiên mà không làm mất tính liên tục.

Ở đây đầu ra là tập hợp các đơn vị đầu ra trong mạng, t_k là giá trị đích của đơn vị k cho ví dụ training d và o_k là đầu ra của đơn vị k cho ví dụ training d .

Trường hợp 1: raini in[^] Quy tắc cho trọng lượng đơn vị đầu ra. Cũng như wji chỉ có thể ảnh hưởng đến phần còn lại của mạng thông qua mạng,, net, chỉ có thể ảnh hưởng đến mạng thông qua o_j . Vì vậy, chúng ta có thể gọi lại quy tắc dây chuyền để viết

Để bắt đầu, hãy xem xét số hạng đầu tiên trong phương trình (4.23)

Đạo hàm $\&(t_k - o_k)^2$ sẽ bằng 0 đối với tất cả các đơn vị đầu ra k ngoại trừ khi $k = j$. Do đó, chúng ta bỏ phép tính tổng theo các đơn vị đầu ra và chỉ cần đặt $k = j$.

Thay thế các biểu thức (4.24) và (4.25) thành (4.23), chúng ta thu được

Và kết hợp điều này với các phương trình (4.21) và (4.22), chúng ta có quy tắc giảm độ dốc ngẫu nhiên cho các đơn vị đầu ra

Trường hợp 2: Quy tắc training cho trọng lượng đơn vị ẩn. Trong trường hợp j là đơn vị nội bộ hoặc ẩn trong mạng, việc rút ra quy tắc training cho wji phải tính đến các cách gián tiếp mà wji có thể ảnh hưởng đến đầu ra của mạng và do đó là E_d . Vì lý do này, chúng ta sẽ thấy hữu ích khi đề cập đến tập hợp tất cả các đơn vị ngay phía dưới đơn vị j trong mạng (tức là tất cả các đơn vị có đầu vào trực tiếp bao gồm đầu ra của đơn vị j). Chúng ta biểu thị tập hợp các đơn vị này bằng

Hạ lưu(j). Lưu ý rằng net_j chỉ có thể ảnh hưởng đến đầu ra mạng (và do đó là E_d) thông qua các đơn vị trong Downstream(j). Vì vậy, chúng ta có thể viết

5.2 Nhận xét về thuật toán Backpropagation

5.2.1 Sự hội tụ và cực tiểu địa phương

Thêm một số hạng động lượng vào quy tắc cập nhật weight như được mô tả trong phương trình (4.18). Động lượng đôi khi có thể thực hiện quy trình giảm độ dốc thông qua các cực tiểu cục bộ hẹp (mặc dù về nguyên tắc, nó cũng có thể mang nó từ cực tiểu toàn cục hẹp đến cực tiểu cục bộ khác!). Sử dụng phương pháp giảm độ dốc ngẫu nhiên thay vì giảm độ dốc thực sự. Như đã thảo luận trong Phần 4.4.3.3, phép tính gần đúng ngẫu

nhân để giảm độ dốc có hiệu quả làm giảm bề mặt lỗi khác nhau cho mỗi ví dụ training, tái

5.2.2 Sức mạnh đại diện của mạng Feedforward

Hãy nhớ rằng các vector weight mạng có thể truy cập bằng cách giảm độ dốc từ các giá trị weight ban đầu có thể không bao gồm tất cả các vector weight có thể có. Hertz và cộng sự. (1991) cung cấp một cuộc thảo luận chi tiết hơn về các kết quả trên.

5.2.3 Giả thuyết tìm kiếm không gian và sai lệch quy nạp

5.2.4 Biểu diễn lớp ẩn

5.3 Đầu vào Đầu ra Đầu vào

10000000
0 1000000 00 100000 00010000 00001000 00000 100 oOOOOo 10 0000000 1

5.4 Ẩn giấu

5.5 Giá trị

.01 .11 .88 + .80 .01 .98 +

5.6 Đầu ra

10000000
**6 1000000 00 100000 000 10000 0000 1000 00000 100
000000 10 0000000 1**

FIGURE 4.7 Biểu diễn lớp ẩn đã học. Mạng 8 x 3 x 8 này được training để tìm hiểu hàm nhận dạng, sử dụng 8 ví dụ training được hiển thị. Sau 5000 kỷ nguyên training, ba giá trị đơn vị ẩn sẽ mã hóa tám đầu vào riêng biệt bằng cách sử dụng mã hóa hiển thị bên phải. Lưu ý nếu các giá trị được mã hóa được làm tròn thành 0 hoặc 1 thì kết quả sẽ là mã hóa nhị phân tiêu chuẩn cho 8 giá trị riêng biệt.

Features có liên quan, để các đơn vị đầu ra có thể sử dụng biểu diễn lớp ẩn này để tính toán các giá trị mục tiêu chính xác.

6.0.1 Tiêu chí khái quát hóa, trang bị quá mức và dừng

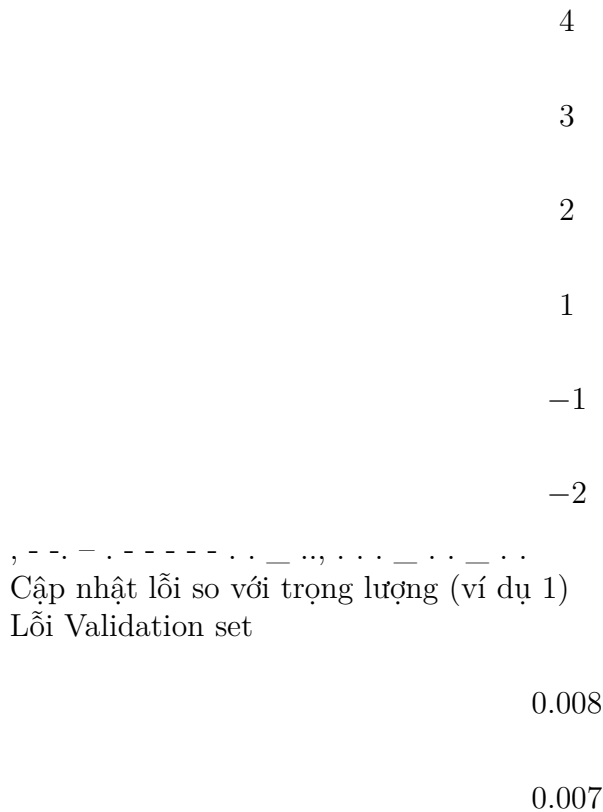
Tổng các sai số bình phương cho mỗi đơn vị đầu ra

Mã hóa đơn vị ẩn cho đầu vào 01000000

FIGURE 4.8

Học mạng 8 x 3 x 8. Biểu đồ trên cùng hiển thị tổng các lỗi bình phương ngày càng tăng đối với từng đơn vị trong số tám đơn vị đầu ra, khi số lần lặp lại training (kỷ nguyên) tăng lên. Biểu đồ ở giữa hiển thị biểu diễn lớp ẩn đang phát triển cho chuỗi đầu vào "01000000." Biểu đồ phía dưới hiển thị weight tăng dần của một trong ba đơn vị ẩn.

Trọng lượng từ đầu vào đến một đơn vị ẩn



7 5000 10000 15000 20000

Số lần cập nhật cân nặng

8 10000 20000 30000 40000 50000 60000

Số lần cập nhật cân nặng

Cập nhật lỗi so với trọng lượng (ví dụ 2)

FIGURE 4.9 Sơ đồ lỗi E là hàm của số lần cập nhật $weight$ đối với hai nhiệm vụ nhận biết robot khác nhau. Trong cả hai trường hợp học, lỗi E trên các ví dụ training đều giảm một cách đơn điệu, vì việc giảm độ dốc sẽ giảm thiểu mức độ lỗi này. Lỗi trên tập hợp các ví dụ "xác thực" riêng biệt thường giảm lúc đầu, sau đó có thể tăng lên do *overfitting* các ví dụ training. Mạng mà *IkeIy* có thể khái quát chính xác nhất cho dữ liệu không nhìn thấy là mạng có sai số thấp nhất so với *validation set*. Lưu ý trong cốt truyện thứ hai, người ta phải cẩn thận để không ngừng training quá sớm khi lỗi *validation set* bắt đầu tăng.

0.07

0.06

Nhìn chung, vấn đề của *overfitting* và cách khắc phục là một vấn đề tế nhị.

Cách tiếp cận *cross-validation* ở trên hoạt động tốt nhất khi có thêm dữ liệu để cung cấp *validation set*. Tuy nhiên, thật không may, vấn đề của *overfitting* là

9 Machine Learning

9.1 Một ví dụ minh họa: Nhận dạng khuôn mặt

9.1.1 Nhiệm vụ

Nhiệm vụ học tập ở đây liên quan đến việc classification hình ảnh khuôn mặt của nhiều người trong các tư thế khác nhau. Hình ảnh của 20 người khác nhau đã được thu thập, bao gồm khoảng 32 hình ảnh cho mỗi người, thay đổi biểu cảm của mỗi người (vui, buồn, tức giận, bình thường), hướng họ nhìn (trái, phải, thẳng về phía trước, lên trên) và liệu họ có đeo kính râm hay không. Như có thể thấy từ các ảnh ví dụ trong Hình 4.10, cũng có sự khác biệt về hậu cảnh phía sau người đó, quần áo người đó mặc và vị trí khuôn mặt của người đó trong ảnh. Tổng cộng, 624 hình ảnh thang độ xám đã được thu thập, mỗi hình ảnh có độ phân giải 120×128 , với mỗi pixel hình ảnh được mô tả bằng giá trị cường độ thang độ xám trong khoảng từ 0 (đen) đến 255 (trắng).

9.2 Weight mạng sau 100 lần lặp qua mỗi mẫu training

FIGURE 4.10 Học neural network nhân tạo để nhận dạng tư thế khuôn mặt. Ở đây, mạng $960 \times 3 \times 4$ được training dựa trên hình ảnh cấp độ xám của khuôn mặt (xem trên cùng), để dự đoán xem một người đang nhìn sang trái, phải, phía trước hay lên trên. Sau khi training trên 260 hình ảnh như vậy, mạng đạt được độ chính xác 90% so với test set riêng biệt. Các weight mạng đã học được hiển thị sau một lần lặp điều chỉnh weight thông qua các ví dụ training và sau 100 lần lặp. Mỗi đơn vị đầu ra (trái, thẳng, phải, lên) có bốn weight, được thể hiện bằng các khối tối (âm) và sáng (dương). Khối ngoài cùng bên trái tương ứng với weight w_g , xác định ngưỡng đơn vị và ba khối bên phải tương ứng với weight trên đầu vào từ ba đơn vị ẩn. Weight từ các pixel hình ảnh vào từng đơn vị ẩn cũng được hiển thị, với mỗi weight được biểu thị ở vị trí của pixel hình ảnh tương ứng.

9.2.1 Lựa chọn thiết kế

Mã hóa đầu ra. ANN phải xuất ra một trong bốn giá trị cho biết hướng mà người đó đang nhìn (trái, phải, lên hoặc thẳng). Lưu ý rằng chúng ta có thể mã hóa classification bốn chiều này bằng cách sử dụng một đơn vị đầu ra duy nhất, gán các đầu ra, chẳng hạn như 0,2,0,4,0,6 và 0,8 để mã hóa bốn giá trị có thể này. Thay vào đó, chúng ta sử dụng bốn đơn vị đầu ra riêng biệt, mỗi đơn vị đại diện cho một trong bốn hướng mặt có thể có, với đầu ra có giá trị cao nhất được lấy làm dự đoán mạng. Điều này thường được gọi là mã hóa đầu ra 1-of-n. Có hai động lực để lựa chọn

Trái, (0,1,0,0) để mã hóa một khuôn mặt nhìn thẳng, v.v. Thay vì giá trị 0 và 1, chúng ta sử dụng các giá trị 0,1 và 0,9, do đó (0,9,0,1,0,1,0,1) là vector đầu ra mục tiêu cho khuôn mặt nhìn sang trái. Lý do tránh các giá trị mục tiêu 0 và 1 là các đơn vị sigmoid không thể tạo ra các giá trị đầu ra này với weight hữu hạn. Nếu chúng ta cố gắng training mạng để phù hợp với các giá trị mục tiêu chính xác là 0 và 1, thì việc giảm độ dốc sẽ buộc các weight tăng lên không giới hạn. Mặt khác, các giá trị 0,1 và 0,9 có thể đạt được bằng cách sử dụng đơn vị sigmoid có weight hữu hạn.

Thuật toán học tập khác parameters. Trong các thí nghiệm học tập này, tốc độ học r được đặt thành 0,3 và động lượng α được đặt thành 0,3. Giá trị thấp hơn cho cả parameters tạo ra độ chính xác tổng quát gần như tương đương, nhưng thời gian training dài hơn. Nếu các giá trị này được đặt quá cao, quá trình training sẽ không hội tụ được

vào mạng có lỗi có thể chấp nhận được trên training set. Giảm độ dốc đầy đủ đã được sử dụng trong tất cả các thử nghiệm này (ngược lại với phép tính gần đúng ngẫu nhiên đối với giảm độ dốc trong thuật toán của Bảng 4.2). Weight mạng trong các đơn vị đầu ra được khởi tạo thành các giá trị ngẫu nhiên nhỏ. Tuy nhiên, weight đơn vị đầu vào được khởi tạo bằng 0, vì điều này mang lại hình ảnh trực quan dễ hiểu hơn nhiều về weight đã học (xem Hình 4.10), mà không có bất kỳ tác động đáng chú ý nào đến độ chính xác tổng quát hóa. Các

Số lần lặp training đã được chọn bằng cách phân vùng dữ liệu có sẵn thành training set và validation set riêng biệt. Giảm độ dốc được sử dụng để giảm thiểu lỗi trên training set và sau mỗi 50 bước giảm độ dốc, hiệu suất của mạng được đánh giá trên validation set. Mạng được chọn cuối cùng là mạng có độ chính xác cao nhất so với validation set. Xem Phần 4.6.5 để biết giải thích và biện minh cho thủ tục này. Độ chính xác được báo cáo cuối cùng (ví dụ: 90% cho mạng trong Hình 4.10) được đo lường qua tập hợp các ví dụ kiểm tra thứ ba không được sử dụng dưới bất kỳ hình thức nào để tác động đến việc training.

9.2.2 Các biểu diễn ẩn đã học

Thật thú vị khi kiểm tra các giá trị weight đã học cho 2899 weight trong mạng. Hình 4.10 mô tả giá trị của từng weight này sau một lần lặp thông qua cập nhật weight cho tất cả các ví dụ training và một lần nữa sau 100 lần lặp.

Gần bằng 0, vì mặt sáng sẽ có xu hướng thẳng hàng với các weight âm lớn trong trường hợp này.

9.3 Các chủ đề nâng cao về mạng nơ ron nhân tạo

9.3.1 Chức năng lỗi thay thế

Ở đây x : biểu thị giá trị của đơn vị đầu vào thứ j cho ví dụ training d .

Vì vậy, 2 là đạo hàm training mô tả giá trị đầu ra mục tiêu

0 Giảm thiểu entropy chéo của mạng đối với các giá trị đích.

9.3.2 Quy trình giảm thiểu lỗi thay thế

9.3.3 Mạng định kỳ

Cho đến thời điểm này, chúng ta chỉ xem xét các cấu trúc liên kết mạng tương ứng với đồ thị có hướng không theo chu kỳ. Mạng lặp lại là artificial neural networks áp dụng cho dữ liệu chuỗi thời gian và sử dụng đầu ra của các đơn vị mạng tại thời điểm t làm đầu vào cho các đơn vị khác tại thời điểm $t + 1$. Bằng cách này, chúng hỗ trợ một dạng chu trình được định hướng trong mạng. Để minh họa, hãy xem xét nhiệm vụ dự đoán chuỗi thời gian để dự đoán giá trị trung bình của thị trường chứng khoán ngày hôm sau $y(t + 1)$ dựa trên các chỉ số kinh tế của ngày hiện tại $x(t)$. Với một chuỗi thời gian gồm những dữ liệu như vậy, một cách tiếp cận rõ ràng là training mạng chuyển tiếp để dự đoán $y(t + 1)$ là đầu ra của nó, dựa trên các giá trị đầu vào $x(t)$. Mạng như vậy được thể hiện trong Hình 4.11(a).

9.4 Tôi 120 Machine Learning

FIGURE 4.11 Mạng định kỳ.

9.4.1 Tự động sửa đổi cấu trúc mạng

Cho đến thời điểm này, chúng ta đã coi neural network learning là bài toán điều chỉnh weight trong cấu trúc đồ thị cố định. Một loạt các phương pháp đã được đề xuất để tăng hoặc giảm linh hoạt số lượng đơn vị mạng và kết nối nhằm cải thiện độ chính xác tổng quát hóa và hiệu quả training.

9.5 Tóm tắt và đọc thêm

Các điểm chính của chương này bao gồm:

Các chức năng đơn giản như XOR không thể được biểu diễn hoặc học bằng mạng perceptron một lớp và hoạt động trên ANNs đã suy thoái trong những năm 1970.

Các phương pháp tiếp cận ANN đã được tích cực khám phá. Sự ra đời của các máy tính rẻ tiền trong cùng thời kỳ này đã cho phép thử nghiệm các thuật toán tính toán chuyên sâu mà không thể khám phá kỹ lưỡng trong những năm 1960.

Một số sách giáo khoa được dành cho chủ đề neural network learning.

Một cuốn sách sớm nhưng vẫn hữu ích về phương pháp học parameter để nhận dạng mẫu là Duda và Hart (1973). Văn bản của Widrow và Stearns (1985) đề cập đến các perceptron và các mạng một lớp liên quan cũng như các ứng dụng của chúng. Rumelhart và McClelland (1986) đã biên soạn một bộ sưu tập các bài báo đã được chỉnh sửa giúp tạo ra sự quan tâm ngày càng tăng đối với các phương pháp này bắt đầu từ giữa những năm 1980. Những cuốn sách gần đây về neural network learning bao gồm Bishop (1996); Chauvin và Rumelhart (1995); Freeman và Skapina (1991); Fu (1994); Hecht-Nielsen (1990); và Hertz và cộng sự. (1991).

10 Bài tập

4.3. Xét hai perceptron được xác định bởi biểu thức ngưỡng $w_0 + w_1 x_1 + w_2 x_2 > 0$.

Đúng hay sai? Perceptron A là tổng quát hơn~han perceptron B. (tổng quát hơn~han được định nghĩa trong Chương 2.)

4.5. Rút ra quy tắc training giảm độ dốc cho một đơn vị có đầu ra 0, trong đó

A b d

101

010

”việc giảm độ dốc có khả năng tìm thấy trọng lượng như vậy không?”

4.10. Hãy xem xét hàm lỗi thay thế được mô tả trong Phần 4.8.1

Suy ra quy tắc cập nhật giảm độ dốc cho định nghĩa này của E. Chứng minh rằng nó có thể được thực hiện bằng cách nhân mỗi weight với một hằng số nào đó trước khi thực hiện cập nhật giảm độ dốc tiêu chuẩn được đưa ra trong Bảng 4.2.

4.11. Áp dụng BACKPROPAGATION vào nhiệm vụ nhận dạng khuôn mặt. Xem World Wide Web

URL <http://www.cs.cmu.edu/~tomlbook.html> để biết chi tiết, bao gồm dữ liệu hình ảnh khuôn mặt, mã BACKPROPAGATION và các tác vụ cụ thể.

Thuật toán của bạn có hội tụ đến giả thuyết lỗi tối thiểu khi các ví dụ dương và âm được phân tách bằng một hình chữ nhật không? Khi họ không? Bạn có

Có vấn đề với cực tiểu địa phương? Thuật toán của bạn so sánh với các phương pháp ký hiệu như thế nào để học các liên từ của ràng buộc feature?

10.1 Tài liệu tham khảo

- Giám mục, C. M. (1996). Neural networks để nhận dạng mẫu. Oxford, Anh: Đại học Oxford
Nhấn.
- Chauvin, Y., & Rumelhart, D. (1995). BACKPROPAGATION: Lý thuyết, kiến trúc và ứng dụng (bộ sưu tập đã được chỉnh sửa). Hillsdale, NJ: Lawrence Erlbaum PGS.
- Churchland, P. S., & Sejnowski, T. J. (1992). Bộ não tính toán. Cambridge, MA: MIT
Nhấn.
- Dietterich, T. G., Hild, H., & Bakiri, G. (1995). So sánh ID3 và BACKPROPAGATION cho
Ảnh xạ văn bản thành giọng nói tiếng Anh. Machine Learning, 18(1), 51-80.
- Duda, R., & Hart, P. (1973). Phân tích mẫu lớp@cation và cảnh. New York: John Wiley &
Con trai.
- Elman, J. L. (1990). Tìm kiếm cấu trúc trong thời gian Khoa học nhận thức, 14, 179-211.
- Fahlman, S., & Lebiere, C. (1990). Kiến trúc học tập CASCADE-CORRELATION (Kỹ thuật Báo cáo CMU-CS-90-100). Khoa Khoa học Máy tính, Đại học Carnegie Mellon, Pittsburgh, PA.
- Huang, W. Y., & Lippmann, R. P. (1988). Mạng lưới thần kinh và classification truyền thống. Trong Anderson (Ed.),
Hệ thống xử lý thông tin thần kinh (trang 387-396).
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L.D.
- LeCun, Y., Denker, J. S., & Solla, S. A. (1990). Tổn thương não tối ưu. Trong D. Touretzky (Ed.),
Những tiến bộ trong Hệ thống xử lý thông tin thần kinh (Tập 2, trang 598-605). San Mateo, CA:
Morgan Kaufmann.
- Phân tích và nhận dạng tài liệu. Montréal, Canada: Hiệp hội máy tính IEEE.
- McCulloch, W. S., & Pitts, W. (1943). Một phép tính logic về những ý tưởng nội tại trong hoạt động thần kinh.
- Bản tin Vật lý sinh học toán học, 5, 115-133.
- Mitchell, T. M., & Thrun, S. B. (1993). Neural network learning dựa trên giải thích để điều khiển robot.
- Trong Hanson, Cowan, & Giles (Eds.), Những tiến bộ trong hệ thống xử lý thông tin thần kinh 5 (trang 287-294). San Francisco: Morgan Kaufmann.
- Moser, M. (1995). Thuật toán BACKPROPAGATION tập trung để nhận dạng mẫu thời gian. TRONG
Y. Chauvin & D. Rumelhart (Eds.), Backpropagation: Lý thuyết, kiến trúc và ứng dụng (trang 137-169). Hillsdale, NJ: Hiệp hội Lawrence Erlbaum.
- Minsky, M., & Papert, S. (1969). Perceptron. Cambridge, MA: Nhà xuất bản MIT.
- Nilsson, N. J. (1965). Máy học tập. New York: Đồi McGraw.
- Parker, D. (1985). Logic học

tập (Báo cáo kỹ thuật MIT TR-47). Trung tâm nghiên cứu MIT ở

Kinh tế tính toán và khoa học quản lý.

Shavlik, J. W., Mooney, R. J., & Towell, G. G. (1991). Các thuật toán học tập tượng trưng và thần kinh:

Một so sánh thực nghiệm Machine Learning, 6(2), 11 1-144.

Hệ thống xử lý thông tin 4 (trang 895-903). San Francisco: Morgan Kaufmann.

Werbos, P. (1975). Beyond regression: Các công cụ mới để dự đoán và phân tích trong khoa học hành vi

(Luận án tiến sĩ). Đại học Harvard.

Widrow, B., & Hoff, M. E. (1960). Mạch chuyển mạch thích ứng. Bản ghi Công ước IRE WESCON,

Widrow, B., & Stearns, S. D. (1985). Xử lý tín hiệu thích ứng Chuỗi xử lý tín hiệu. Englewood

Vách đá, NJ: Hội trường Prentice.

Zometzer, S. F., Davis, J. L., & Lau, C. (1994). Giới thiệu về networks thần kinh và điện tử

(bộ sưu tập đã chỉnh sửa) (tái bản lần thứ 2). New York: Nhà xuất bản học thuật.

10.2 Đánh giá giả thuyết

Đánh giá theo kinh nghiệm tính chính xác của các giả thuyết là nền tảng của machine learning. Chương này trình bày phần giới thiệu về các phương pháp thống kê để ước tính độ chính xác của giả thuyết, tập trung vào ba câu hỏi. Đầu tiên, với độ chính xác quan sát được của một giả thuyết đối với một mẫu dữ liệu hạn chế, estimate này có độ chính xác như thế nào so với các ví dụ bổ sung? Thứ hai, giả thuyết này tốt hơn giả thuyết khác về một số mẫu dữ liệu, nói chung giả thuyết này có khả năng chính xác hơn như thế nào? Thứ ba, khi dữ liệu bị hạn chế, cách tốt nhất để sử dụng dữ liệu này để tìm hiểu một giả thuyết và tính chính xác của nó là estimate? Bởi vì các mẫu dữ liệu hạn chế có thể trình bày sai sự phân bố dữ liệu chung nên việc ước tính độ chính xác thực sự từ các mẫu đó có thể gây hiểu nhầm. Các phương pháp thống kê, cùng với các giả định về phân phối dữ liệu cơ bản, cho phép người ta xác định sự khác biệt giữa độ chính xác quan sát được trên mẫu dữ liệu có sẵn và độ chính xác thực sự trên toàn bộ phân phối dữ liệu.

10.3 Động lực

Trong nhiều trường hợp, điều quan trọng là phải đánh giá hiệu quả của các giả thuyết đã học một cách chính xác nhất có thể. Một lý do đơn giản là để hiểu có nên sử dụng giả thuyết hay không. Ví dụ, khi học từ cơ sở dữ liệu có kích thước giới hạn cho thấy hiệu quả của các phương pháp điều trị y tế khác nhau, điều quan trọng là phải hiểu chính xác nhất có thể tính chính xác của các giả thuyết đã học. Lý do thứ hai là việc đánh giá các giả thuyết là một phần không thể thiếu trong nhiều phương pháp học tập. Ví dụ: trong quá trình cắt tỉa decision trees sau cắt tỉa để tránh overfitting, chúng ta phải đánh giá